

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM : 123240122
123240138
NAMA : Ichsan Wahyu Priambudi
Muhamad Ifsal Fadhilah
PLUG : IF-G
NAMA ASISTEN : Farhannivta Ramadhana
Muhammad Ruhul Jadid

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Ichsan Wahyu Priambudi	123240122
Muhamad Ifsal Fadhilah	123240138

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan
Pada Tanggal:

Asisten Praktikum

Farhannivta Ramadhana
NIM. 123230139

Asisten Praktikum

Muhammad Ruhul Jadid
NIM. 123230046

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul Sistem Cerdas Pendukung Keputusan Pemilihan laptop untuk *Gaming*. Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 28 Mei 2026

Penyusun 1,

Penyusun 2,

Ichsan Wahyu Priambudi

NIM. 123240122

Muhamad Ifsal Fadhilah

NIM. 123240138

DAFTAR ISI

HALAMAN PENGESAHAN.....	ii
KATA PENGANTAR.....	iii
DAFTAR ISI.....	iv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Masalah.....	1
1.2 Tujuan Proyek Akhir.....	1
1.3 Manfaat Proyek Akhir.....	1
BAB II PEMBAHASAN.....	2
2.1 Dasar Teori.....	2
2.2 Skenario Kasus & Dataset.....	3
2.3 Pemodelan Sistem Pendukung Keputusan.....	3
2.4 Perhitungan & Algoritma.....	4
2.5 Implementasi & Tampilan Sistem.....	4
BAB III JADWAL Pengerjaan DAN PEMBAGIAN TUGAS.....	6
3.1 Jadwal Pengerjaan.....	6
3.2 Pembagian Tugas.....	6
BAB IV KESIMPULAN DAN SARAN.....	7
4.2 Kesimpulan.....	7
4.3 Saran.....	7
DAFTAR PUSTAKA.....	8

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan teknologi industri *game* yang pesat menuntut ketersediaan perangkat keras yang memadai. Banyak konsumen yang kebingungan memilih laptop gaming yang tepat karena banyaknya variasi spesifikasi (CPU, GPU, RAM, dan *Storage*) yang ditawarkan di pasaran dengan rentang harga yang beragam. Sebagai contoh, seorang pengguna yang saat ini menggunakan laptop entry-level atau menengah untuk komputasi harian, seringkali merasa kesulitan ketika harus beralih dan memilih laptop baru yang dikhususkan untuk kebutuhan beban kerja berat seperti *gaming*. Dibutuhkan sebuah Sistem Pendukung Keputusan (SPK) untuk memberikan rekomendasi laptop terbaik secara objektif berdasarkan kriteria-kriteria yang diinginkan oleh konsumen, sehingga pembelian menjadi lebih tepat sasaran dan efisien.

1.2 Tujuan Proyek Akhir

1. Mengimplementasikan metode Weighted Product (WP) ke dalam sebuah Sistem Pendukung Keputusan.
2. Membangun aplikasi berbasis web interaktif menggunakan Python dan library Streamlit untuk memilih laptop gaming terbaik berdasarkan spesifikasi dan harga.

1.3 Manfaat Proyek Akhir

1. Membantu calon pembeli (khususnya gamer dan mahasiswa) untuk mengambil keputusan pembelian laptop yang rasional berdasarkan komputasi data.
2. Memberikan antarmuka (UI) yang mudah digunakan bagi pengguna awam untuk melihat peringkat laptop terba

BAB II PEMBAHASAN

2.1 Dasar Teori

Metode Weighted Product (WP) adalah salah satu metode penyelesaian dalam sistem pendukung keputusan yang menggunakan teknik perkalian untuk menghubungkan rating atribut, di mana rating setiap atribut harus dipangkatkan terlebih dahulu dengan bobot kriteria yang bersangkutan. Proses ini diawali dengan melakukan perbaikan atau normalisasi bobot keputusan

Rumus untuk melakukan perbaikan bobot awal menjadi bobot ternormalisasi adalah sebagai berikut:

$$w_j = \frac{w_j}{\sum_{j=1}^n w_j}$$

Di mana W_j merupakan bobot ternormalisasi dan w_j melambangkan bobot kepentingan awal (skala 1-5) yang dimasukkan oleh pengguna.

Setelah nilai bobot diperbaiki, langkah berikutnya adalah menghitung nilai preferensi untuk masing-masing alternatif yang direpresentasikan melalui Vektor S_i dengan rumus matematika:

$$S_i = \prod_{j=1}^n (X_{ij} + \epsilon)^{k_j \cdot w_j}$$

Di mana X_{ij} merupakan nilai matriks keputusan untuk alternatif i pada kriteria j . Variabel k_j bernilai 1 jika kriteria bertipe keuntungan (*benefit*) dan bernilai -1 jika kriteria bertipe biaya (*cost*). Untuk mengantisipasi terjadinya kesalahan komputasi matematis akibat nilai data bernilai nol, ditambahkan konstanta pengaman berupa epsilon ($\epsilon = 1e - 9$).

Langkah terakhir adalah menghitung nilai Vektor V_i yang digunakan untuk mengukur nilai preferensi relatif dari suatu alternatif terhadap seluruh total alternatif yang ada dengan rumus:

$$S_i = \frac{S_i}{\sum_{i=1}^m S_i}$$

Alternatif yang memiliki nilai Vektor V_i terbesar akan diurutkan di posisi teratas sebagai rekomendasi laptop terbaik.

2.2 Skenario Kasus & Dataset

Konsumen atau pengguna umum yang membutuhkan perangkat komputer jinjing (laptop) performa tinggi untuk kebutuhan bermain *game* (*gaming*). Pengambil keputusan dapat mengatur tingkat kepentingan dari setiap aspek spesifikasi secara dinamis melalui komponen *sidebar* interaktif. Memberikan draf urutan peringkat rekomendasi model laptop dari nilai preferensi tertinggi hingga terendah berdasarkan pencocokan data teknis secara objektif.

Sistem ini tidak hanya bergantung pada satu sumber data, melainkan mengintegrasikan tiga buah dataset yang saling terhubung di dalam folder dokumen:

- a. *laptop.csv*: Berisi 200 data sampel alternatif laptop yang memuat data teks manufaktur (Brand), model fisik (Model), kapasitas memori acak (RAM), kapasitas penyimpanan (Storage), serta harga jual perangkat (Price_USD).
- b. *CPU.csv*: Berisi data performa prosesor di pasar dunia, di mana indikator yang diambil adalah nilai benchmark performa multi-inti (multiScore).
- c. *GPU.csv*: Berisi data performa kartu grafis eksternal, di mana indikator performa yang diambil adalah nilai skor benchmark pengolah grafis tiga dimensi (3DMark).

Proses penggabungan data (*data merging*) dilakukan dengan mengekstrak 3 kata pertama pada nama CPU serta 1 kata pertama pada nama GPU dari tabel alternatif, kemudian dicocokkan menggunakan metode *left join* ke dataset benchmark untuk menghasilkan ringkasan parameter data final yang utuh.

2.3 Pemodelan Sistem Pendukung Keputusan

Dalam implementasi metode WP pada sistem ini, kriteria penilaian dan objek alternatif dimodelkan secara terstruktur sebagai berikut:

- a. Alternatif (A_i): Model-model laptop spesifik yang datanya berhasil dibersihkan dan digabungkan secara lengkap dari dataset final (df_datafinal).
- b. Kriteria (C_j): Menggunakan 5 buah kriteria teknis utama sebagai dasar penilaian bobot keputusan:
 - 1) C1 - Performa CPU (multiScore): Kriteria *Benefit* (Semakin besar nilai skor benchmark multi-inti prosesor, semakin baik).
 - 2) C2 - Performa GPU (3DMark): Kriteria *Benefit* (Semakin tinggi skor pengujian grafis 3DMark kartu grafis, semakin cocok untuk *gaming*).
 - 3) C3 - Kapasitas RAM: Kriteria *Benefit* (Nilai kapasitas RAM dalam satuan Gigabyte setelah teks satuannya dibersihkan, semakin besar semakin baik).
 - 4) C4 - Kapasitas Storage: Kriteria *Benefit* (Nilai total ruang penyimpanan yang disetarakan dalam satuan Gigabyte, di mana 1 Terabyte dikonversi menjadi 1024 GB).
 - 5) C5 - Harga Laptop (Price_USD): Kriteria *Cost* (Nilai nominal harga jual laptop dalam mata uang USD, semakin kecil nilainya semakin menguntungkan konsumen).

Tingkat kepentingan kriteria ditentukan langsung oleh pengguna menggunakan komponen penilai *slider* dari rentang angka 1 sampai 5:

- a. Angka 1: Sangat Tidak Penting
- b. Angka 2: Tidak Penting
- c. Angka 3: Cukup Penting
- d. Angka 4: Penting
- e. Angka 5: Sangat Penting

2.4 Perhitungan & Algoritma

Tahapan komputasi pengolahan data serta algoritma matematis *Weighted Product* di dalam sistem ini mengikuti urutan kerja sebagai berikut:

- a. **Pra-pemrosesan Data (Data Cleaning):** Sistem memanggil fungsi `konversi_storage` untuk membersihkan teks satuan "TB" atau "GB" menjadi nilai murni, serta fungsi `konversi_ram` untuk mengekstrak karakter angka dari kolom teks RAM agar bisa dibaca sebagai tipe data numerik mengambang (*float*). Nilai harga pada kolom `Price_USD` juga dipastikan bertipe data numerik.
- b. **Penyusunan Matriks Keputusan:** Mengambil nilai baris kriteria dari kolom ['multiScore', '3DMark', 'RAM', 'Storage', 'Price_USD'] untuk diubah menjadi struktur array multi-dimensi menggunakan perintah `to_numpy()`.
- c. **Normalisasi Bobot Pengguna:** Membaca array bobot awal pengguna dari komponen input interaktif. Nilai bobot tersebut dibagi dengan hasil penjumlahan total bobot awal untuk mendapatkan variabel `norm_bobot`. Array penanda jenis kriteria dideklarasikan sebagai `k = np.array([1, 1, 1, 1, -1])`, di mana indeks terakhir bernilai negatif karena variabel Harga bertindak sebagai kriteria *cost*.
- d. **Kalkulasi Vektor S:** Seluruh elemen dalam matriks keputusan dipangkatkan dengan hasil perkalian `k * norm_bobot`, kemudian dilakukan operasi perkalian berantai antar kolom kriteria per baris menggunakan fungsi `np.prod(..., axis=1)` guna memperoleh nilai Vektor S.
- e. **Kalkulasi Vektor V:** Nilai baris Vektor S yang didapatkan dibagi dengan total jumlahan agregat dari seluruh nilai elemen Vektor S untuk menghasilkan nilai preferensi akhir berupa nilai Vektor V.
- f. **Perankingan Akhir:** Data dipetakan kembali ke dalam struktur tabel utama, lalu diurutkan secara menurun (*descending*) berdasarkan kolom `Vektor_V` untuk memunculkan rekomendasi perangkat dari peringkat pertama hingga terakhir.

2.5 Implementasi & Tampilan Sistem

(isi dengan source code program)

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

st.set_page_config(page title="pemilihan laptop - WP", layout="wide")
```



```

def konversi_storage(x):
    text = str(x).strip().upper()
    angka_text = "".join([char for char in text if char.isdigit()])
    if not angka_text :
        return 0
    angka = float(angka_text)

    if "TB" in text:
        return angka * 1024
    return angka

def konversi_ram(x):
    text = str(x).strip().upper()
    angka_text = "".join([char for char in text if char.isdigit()])
    if not angka_text:
        return 0
    return float(angka_text)

def konversi_Price(x):
    return float(x)

#sidebar
with st.sidebar:

#page
    page = st.selectbox("pilih halaman",[
        "Data RAW",
        "Data CPU",
        "Data GPU",
        "WP",
        "Visualisasi"
    ])

    st.markdown("___")
    st.header("Pengaturan Bobot Kriteria")
    st.markdown("Atur bobot dengan skala 1-5")
    st.markdown("1: Sangat Tidak Penting \n2: Tidak Penting \n3: Cukup  
Penting \n4: Penting \n5: Sangat Penting")

#.slider
    bobot_cpu = st.slider("Bobot CPU (Benefit)", 1, 5, 3)
    bobot_gpu = st.slider("Bobot GPU (Benefit)", 1, 5, 3)
    bobot_ram = st.slider("Bobot RAM (Benefit)", 1, 5, 3)
    bobot_storage = st.slider("Bobot Storage (Benefit)", 1, 5, 3)
    bobot_harga = st.slider("Bobot Harga (Benefit)", 1, 5, 3)

#table alternatif
if page == "Data RAW" :
    st.title("📊 Sistem Pendukung Keputusan Pemilihan Laptop")
    st.markdown("Aplikasi ini menggunakan **Metode Weighted Product (WP)**  
untuk merekomendasikan laptop terbaik berdasarkan kriteria.")

    st.markdown("### 📋 Kriteria Evaluasi Sistem")

```

```

    st.write("Berikut adalah kriteria yang digunakan dalam perhitungan
beserta sifatnya (Benefit/Cost):")

# Membuat 5 kolom untuk 5 kriteria
col1, col2, col3, col4, col5 = st.columns(5)

with col1:
    with st.container(border=True):
        st.markdown("<b style='color: #2ecc71; font-size: 17px;'>🖥️
CPU</b>", unsafe_allow_html=True)
        st.markdown("<span style='background-color: #1e3a1e; color:
#2ecc71; padding: 2px 8px; border-radius: 5px; font-size: 12px;
font-weight: bold;'>BENEFIT</span>", unsafe_allow_html=True)
        st.markdown("<p style='font-size: 13px; margin-top: 8px;
color: #cbd5e1;'>Diukur dari nilai <i>multiScore</i> benchmark CPU.
Semakin tinggi semakin baik.</p>", unsafe_allow_html=True)

    with col2:
        with st.container(border=True):
            st.markdown("<b style='color: #2ecc71; font-size: 17px;'>🎮
GPU</b>", unsafe_allow_html=True)
            st.markdown("<span style='background-color: #1e3a1e; color:
#2ecc71; padding: 2px 8px; border-radius: 5px; font-size: 12px;
font-weight: bold;'>BENEFIT</span>", unsafe_allow_html=True)
            st.markdown("<p style='font-size: 13px; margin-top: 8px;
color: #cbd5e1;'>Diukur dari skor <i>3DMark</i> grafis. Semakin tinggi
semakin bertenaga.</p>", unsafe_allow_html=True)

    with col3:
        with st.container(border=True):
            st.markdown("<b style='color: #2ecc71; font-size: 17px;'>⚡
RAM</b>", unsafe_allow_html=True)
            st.markdown("<span style='background-color: #1e3a1e; color:
#2ecc71; padding: 2px 8px; border-radius: 5px; font-size: 12px;
font-weight: bold;'>BENEFIT</span>", unsafe_allow_html=True)
            st.markdown("<p style='font-size: 13px; margin-top: 8px;
color: #cbd5e1;'>Kapasitas memori (GB). Semakin besar mendukung
multitasking berat.</p>", unsafe_allow_html=True)

    with col4:
        with st.container(border=True):
            st.markdown("<b style='color: #2ecc71; font-size: 17px;'>💾
Storage</b>", unsafe_allow_html=True)
            st.markdown("<span style='background-color: #1e3a1e; color:
#2ecc71; padding: 2px 8px; border-radius: 5px; font-size: 12px;
font-weight: bold;'>BENEFIT</span>", unsafe_allow_html=True)
            st.markdown("<p style='font-size: 13px; margin-top: 8px;
color: #cbd5e1;'>Kapasitas penyimpanan data (GB). Semakin besar semakin
lapang.</p>", unsafe_allow_html=True)

    with col5:
        with st.container(border=True):
            st.markdown("<b style='color: #e74c3c; font-size: 17px;'>💰
Harga</b>", unsafe_allow_html=True)
            st.markdown("<span style='background-color: #3a1e1e; color:
#e74c3c; padding: 2px 8px; border-radius: 5px; font-size: 12px;
font-weight: bold;'>COST</span>", unsafe_allow_html=True)

```

```

        st.markdown("<p style='font-size: 13px; margin-top: 8px; color: #cbd5e1;'>Harga laptop dalam USD. Semakin murah/kecil nilainya semakin baik.</p>", unsafe_allow_html=True)

    st.markdown("<br>", unsafe_allow_html=True)

    st.write("Data Alternatif Laptop")
    df_laptop = pd.read_csv('dataset/laptop.csv')
    df_claptop = df_laptop
    df_laptop.drop(columns=["Laptop_ID"]+["Performance_Level"])
    df_claptop.index = df_claptop.index + 1
    st.dataframe(df_claptop)
    st.session_state["df_rawlaptop"] = df_claptop

#table CPU
if page == "Data CPU" :
    if "df_rawlaptop" in st.session_state:
        df_claptop = st.session_state["df_rawlaptop"]

        st.title("🔍 Analisis & Pemrosesan Data CPU")
        st.markdown("Halaman ini mengelola kecocokan data spesifikasi prosesor antara unit alternatif laptop dengan skor benchmark performa multithreading.")

        # --- BAGIAN UI: KARTU INFORMASI KRITERIA CPU ---
        with st.container(border=True):
            col_icon, col_desc = st.columns([1, 11])
            with col_icon:
                st.markdown("<h1 style='text-align: center; margin: 0;'>⚙️</h1>", unsafe_allow_html=True)
            with col_desc:
                st.markdown("<b style='color: #2ecc71; font-size: 18px;'>Kriteria Utama: CPU Performance</b>", unsafe_allow_html=True)
                st.markdown("<span style='background-color: #1e3a5e; color: #2ecc71; padding: 3px 10px; border-radius: 5px; font-size: 12px; font-weight: bold;'>BENEFIT CRITERION</span>", unsafe_allow_html=True)
                st.markdown(
                    "<p style='font-size: 14px; margin-top: 8px; color: #cbd5e1; line-height: 1.4;'>
                        Performa CPU diukur berdasarkan standarisasi nilai <b>multiScore</b> dari data benchmark. "
                        "Semakin tinggi skor <i>multiScore</i>, maka kemampuan komputasi paralel, rendering, dan multitasking "
                        "pada laptop akan dinilai semakin baik dan mendapatkan prioritas utama dalam pembobotan WP."
                    "</p>",
                    unsafe_allow_html=True
                )

        st.markdown("<br>", unsafe_allow_html=True)

        df_cpu = pd.read_csv('dataset/CPU.csv')
        df_cpu["CPU"] = df_cpu["manufacturer"].astype(str) + " " + df_cpu["namaCPU"].astype(str)
        df_ccpu = df_cpu.drop(columns=["manufacturer", "namaCPU", "singleScore", "cores", "threads", "baseClock", "turboClock", "type"])
        kolom_baru_cpu = ["CPU", "multiScore"]
        df_ccpu = df_ccpu[kolom_baru_cpu]
        #df_laptopcpu = pd.merge(df_claptop, df_ccpu, on = "CPU")

```

```

#st.dataframe(df_laptopcpu)

#tablelaptopcpu
df_claptop["CPU_Clean"] =
df_claptop["CPU"].astype(str).str.split().str[0:3].str.join(' ')
df_ccpu["CPU_Clean"] =
df_ccpu["CPU"].astype(str).str.split().str[0:3].str.join(' ')
df_laptopcpu = pd.merge(df_claptop, df_ccpu, on="CPU_Clean",
how="left")

df_dislaptopcpu =
df_laptopcpu.drop(columns=["CPU_Clean", "Brand", "RAM", "GPU", "Price_USD", "C
PU_x", "Storage"])
df_dislaptopcpu = df_dislaptopcpu.rename(columns={"CPU_y": "CPU"})
df_dislaptopcpu["multiScore"] =
pd.to_numeric(df_dislaptopcpu["multiScore"], errors="coerce")
df_sortcpu = df_dislaptopcpu.sort_values(by="multiScore",
ascending=False)
df_fixlaptopcpu = df_sortcpu.drop_duplicates(subset="Model",
keep="first")
df_fixlaptopcpu = df_fixlaptopcpu.reset_index(drop=True)
df_fixlaptopcpu.index = df_fixlaptopcpu.index + 1
st.session_state["df_fixlaptopcpu"] = df_fixlaptopcpu

# --- BAGIAN UI: INTERAKTIF TABS UNTUK MENAMPILKAN TABEL ---
tab1, tab2 = st.tabs([" Hasil Pemetaan Laptop & CPU", "
Dataset Raw CPU Benchmark"])

with tab1:
    st.markdown("####  Hasil Akhir Pemrosesan Nilai CPU Laptop")
    st.caption("Data alternatif laptop yang telah sukses dipetakan
dengan nilai multiScore CPU yang telah diurutkan dari skor tertinggi:")
    st.dataframe(df_fixlaptopcpu, use_container_width=True)

with tab2:
    st.markdown("####  Seluruh Database CPU Benchmark")
    st.caption("Referensi mentah tabel database performa CPU
berdasarkan data benchmark global:")
    st.dataframe(df_ccpu.drop(columns=["CPU_Clean"],
errors="ignore"), use_container_width=True)

else:
    st.warning("error")

#tablegpu
if page == "Data GPU":
    if "df_rawlaptop" in st.session_state:
        df_claptop = st.session_state["df_rawlaptop"]

        st.title(" Analisis & Pemrosesan Data GPU")
        st.markdown("Halaman ini mengelola kecocokan data spesifikasi
kartu grafis (GPU) antara unit alternatif laptop dengan skor benchmark
performa olah grafis.")

# --- BAGIAN UI: KARTU INFORMASI KRITERIA GPU ---
with st.container(border=True):
    col_icon, col_desc = st.columns([1, 11])
    with col_icon:

```

```

        st.markdown("<h1 style='text-align: center; margin: 0;'>🎮</h1>", unsafe_allow_html=True)
        with col_desc:
            st.markdown("<b style='color: #2ecc71; font-size: 18px;'>Kriteria Utama: GPU Performance</b>", unsafe_allow_html=True)
            st.markdown("<span style='background-color: #1e3a1e; color: #2ecc71; padding: 3px 10px; border-radius: 5px; font-size: 12px; font-weight: bold;'>BENEFIT CRITERION</span>", unsafe_allow_html=True)
            st.markdown(
                "<p style='font-size: 14px; margin-top: 8px; color: #cbd5e1; line-height: 1.4;'>
                    \"Performa kartu grafis diukur berdasarkan standarisasi nilai skor <b>3DMark (G3Dmark)</b>. \"
                    \"Semakin tinggi nilai skor grafis ini, maka kemampuan rendering 3D, gaming, editing video, \"
                    \"serta pemrosesan AI pada laptop akan dinilai semakin bertenaga dan mendapatkan prioritas utama dalam pembobotan WP.\"
                </p>",
                unsafe_allow_html=True
            )

        st.markdown("<br>", unsafe_allow_html=True)

        st.write("Nilai GPU (3DMark)")
        df_gpu = pd.read_csv('dataset/GPU.csv')
        df_cgpu = df_gpu.drop(columns=[
            "G2Dmark", "price", "gpuValue", "TDP", "powerPerformance", "testDate", "category"
        ])
        df_cgpu = df_cgpu.rename(columns={"gpuName": "GPU", "G3Dmark": "3DMark"})
        df_cgpu.index = df_cgpu.index + 1
        #df_laptopcpu = pd.merge(df_claptop, df_ccpu, on = "CPU")
        #st.dataframe(df_cgpu)
        #st.dataframe(df_laptopcpu)

        st.write("Nilai Laptop dan GPU")

        df_claptop["GPU_Clean"] = df_claptop["GPU"].astype(str).str.split().str[0:1].str.join(' ')
        df_cgpu["GPU_Clean"] = df_cgpu["GPU"].astype(str).str.split().str[0:1].str.join(' ')
        df_laptopgpu = pd.merge(df_claptop, df_cgpu, on="GPU_Clean", how="left")

        df_dislaptopgpu = df_laptopgpu.drop(columns=["GPU_Clean", "Brand", "RAM", "CPU", "Price_USD", "GPU_x", "Storage", "CPU_Clean"])
        df_dislaptopgpu = df_dislaptopgpu.rename(columns={"GPU_y": "GPU"})
        df_dislaptopgpu["3DMark"] = pd.to_numeric(df_dislaptopgpu["3DMark"], errors="coerce")
        df_sortgpu = df_dislaptopgpu.sort_values(by="3DMark", ascending=False)
        df_fixlaptopgpu = df_sortgpu.drop_duplicates(subset="Model", keep="first")
        df_fixlaptopgpu = df_fixlaptopgpu.reset_index(drop=True)
        df_fixlaptopgpu.index = df_fixlaptopgpu.index + 1
        #st.dataframe(df_fixlaptopgpu)
        st.session_state["df_fixlaptopgpu"] = df_fixlaptopgpu

        # --- BAGIAN UI: INTERAKTIF TABS UNTUK MENAMPILKAN TABEL ---

```

```

    tab1, tab2 = st.tabs(["📊 Hasil Pemetaan Laptop & GPU", "📁 Dataset Raw GPU Benchmark"])

    with tab1:
        st.markdown("#### 🎯 Hasil Akhir Pemrosesan Nilai GPU Laptop")
        st.caption("Data alternatif laptop yang telah sukses dipetakan dengan nilai skor grafis 3DMark yang telah diurutkan dari skor tertinggi:")
        st.dataframe(df_fixlaptopgpu, use_container_width=True)

    with tab2:
        st.markdown("#### 📄 Seluruh Database GPU Benchmark")
        st.caption("Referensi mentah tabel database performa kartu grafis (GPU) berdasarkan data benchmark global 3DMark:")
        st.dataframe(df_cgpu.drop(columns=["GPU_Clean"], errors="ignore"), use_container_width=True)

if page == "WP" :
    if "df_rawlaptop" in st.session_state and "df_fixlaptopcpu" in st.session_state and "df_fixlaptopgpu" in st.session_state:
        df_claptop = st.session_state["df_rawlaptop"]
        df_fixlaptopcpu = st.session_state["df_fixlaptopcpu"]
        df_fixlaptopgpu = st.session_state["df_fixlaptopgpu"]

        #st.dataframe(df_claptop)
        #st.dataframe(df_fixlaptopcpu)
        #st.dataframe(df_fixlaptopgpu)
        st.title("📋 Proses Perhitungan Metode Weighted Product (WP)")
        st.markdown("Halaman ini menampilkan transparansi kalkulasi matematis Multi-Criteria Decision Making (MCDM) menggunakan algoritma Weighted Product.")
        #datafinal
        with st.expander("🔧 Langkah 1: Penggabungan & Transformasi Tipe Data", expanded=False):
            st.markdown("#### **Penggabungan Tabel Spesifikasi & Nilai Benchmark**")
            st.caption("Menggabungkan dataset laptop utama dengan nilai multiScore CPU dan 3DMark GPU berdasarkan pencocokan nama komponen:")

            st.write("Data Final")
            df_step1 = pd.merge(df_claptop, df_fixlaptopcpu, on="Model", how="left")
            df_stepfinal = pd.merge(df_step1, df_fixlaptopgpu, on="Model", how="left")
            df_final = df_stepfinal.dropna()

            df_datafinal = df_final.drop(columns=["Brand", "CPU_x", "GPU_x", "CPU_Clean", "GPU_Clean", "CPU_y", "GPU_y"])
            st.dataframe(df_datafinal)

        #konversi semua ke float
        st.markdown("#### **Hasil Konversi Nilai Kriteria Menjadi Numerik (Float)**")
        st.caption("Mengubah teks kapasitas Storage (GB), RAM (GB), dan Harga menjadi angka koma agar bisa diproses secara matematis:")

        df_datafinal["Storage"] = df_datafinal["Storage"].apply(konversi_storage)

```

```

df_datafinal["RAM"] = df_datafinal["RAM"].apply(konversi_ram)
df_datafinal["Price_USD"] = df_datafinal["Price_USD"].apply(konversi_Price)
st.dataframe(df_datafinal)
#df_datafinal.info()

#WP method

with st.expander("⚖️ Langkah 2: Perbaikan Bobot Kriteria (Normalisasi)", expanded=False):
    st.markdown("#### **Tabel Bobot Kriteria**")
    st.markdown(
        "Sesuai kaidah Weighted Product, bobot awal dari slider diperbaiki sehingga total seluruh bobot bernilai 1 ( $\sum w = 1$ ). "
        "Pangkat bernilai positif jika kriteria bersifat **Benefit**, dan negatif jika kriteria bersifat **Cost**."
    )

    nama_kriteria = ("CPU", "GPU", "RAM", "Storage", "Harga")
    bobot_awal = np.array([bobot_cpu, bobot_gpu, bobot_ram, bobot_storage, bobot_harga])
    k = np.array([1, 1, 1, 1, -1])

    total_bobot_awal = np.sum(bobot_awal)
    norm_bobot = bobot_awal / total_bobot_awal

    df_bobot = pd.DataFrame({
        "Nama Kriteria": nama_kriteria,
        "Bobot Kriteria": bobot_awal,
        "Bobot Ternormalisasi": np.round(norm_bobot, 4)
    })
    df_bobot.index = df_bobot.index + 1
    st.dataframe(df_bobot)

    kolom_kriteria = ["multiScore", "3DMark", "RAM", "Storage", "Price_USD"]
    matrix_x = df_datafinal[kolom_kriteria].to_numpy()
    #RumusWP

    vektor_s = np.prod((matrix_x + 1e-9) ** (k * norm_bobot), axis=1)

    vektor_v = vektor_s / np.sum(vektor_s)

    df_wp_result = df_datafinal.copy()
    df_wp_result["Vektor_S"] = vektor_s
    df_wp_result["Vektor_V"] = vektor_v

    with st.expander("🧮 Langkah 3: Matriks Perhitungan Vektor S", expanded=False):
        st.markdown("#### **Nilai Kelayakan Komparatif (Vektor S)**")
        st.markdown("Rumus perkalian matriks terhadap pangkat bobot kriteria baru:  $S_i = \prod_{j=1}^n X_{ij}^{w_j}$ ")
        st.dataframe(df_wp_result[["Model", "Vektor_S"]], use_container_width=True)

        st.markdown("<br>", unsafe_allow_html=True)

```

```

df_ranking = df_wp_result.sort_values(by="Vektor_V",
ascending=False).reset_index(drop=True)
df_ranking.index = df_ranking.index + 1

#st.success("Hasil Rekomendasi Peringkat Laptop: ")

kolom_tampilan_akhir = ["Model", "RAM", "Storage", "Price_USD",
"multiScore", "3DMark", "Vektor_V"]
#st.dataframe(df_ranking[kolom_tampilan_akhir])
st.session_state["df_ranking"] = df_ranking

st.markdown("### 🏆 Hasil Akhir Rekomendasi Laptop Terbaik")
st.write("Daftar alternatif laptop yang telah diurutkan
berdasarkan nilai preferensi tertinggi (Vektor V):")

# Box Highlight Juara Peringkat 1
laptop_juara = df_ranking.iloc[0]
st.info(
    f"🎯 **Rekomendasi Utama:** Berdasarkan pembobotan yang kamu
atur, laptop terbaik jatuh kepada "
    f"**{laptop_juara['Model']}** dengan skor preferensi akhir
tertinggi yaitu **{laptop_juara['Vektor_V']:.4f}**."
)

kolom_tampilan_akhir = ["Model", "RAM", "Storage", "Price_USD",
"multiScore", "3DMark", "Vektor_V"]
st.dataframe(df_ranking[kolom_tampilan_akhir],
use_container_width=True)

if page == "Visualisasi":
    st.title("📊 Visualisasi Hasil Rekomendasi")

    # Cek apakah data pendukung dari page CPU & GPU sudah siap di
session_state
    if "df_rawlaptop" in st.session_state and "df_fixlaptopcpu" in
st.session_state and "df_fixlaptopgpu" in st.session_state:
        df_claptop = st.session_state["df_rawlaptop"]
        df_fixlaptopcpu = st.session_state["df_fixlaptopcpu"]
        df_fixlaptopgpu = st.session_state["df_fixlaptopgpu"]

        # 1. Proses kalkulasi ulang WP secara background agar grafik
selalu sinkron dengan slider
        df_step1 = pd.merge(df_claptop, df_fixlaptopcpu, on="Model",
how="left")
        df_stepfinal = pd.merge(df_step1, df_fixlaptopgpu, on="Model",
how="left")
        df_final = df_stepfinal.dropna()
        df_datafinal = df_final.drop(columns=["Brand", "CPU_x", "GPU_x",
"CPU_Clean", "GPU_Clean", "CPU_y", "GPU_y"])

        # Konversi tipe data
df_datafinal["Storage"] =
df_datafinal["Storage"].apply(konversi_storage)
df_datafinal["RAM"] = df_datafinal["RAM"].apply(konversi_ram)
df_datafinal["Price_USD"] =
df_datafinal["Price_USD"].apply(konversi_Price)

# Hitung Bobot & Vektor S-V

```



```

        bobot_awal = np.array([bobot_cpu, bobot_gpu, bobot_ram,
bobot_storage, bobot_harga])
        k = np.array([1, 1, 1, 1, -1])

        total_bobot_awal = np.sum(bobot_awal)
        norm_bobot = bobot_awal / total_bobot_awal

        kolom_kriteria = ["multiScore", "3DMark", "RAM", "Storage",
"Price_USD"]
        matrix_x = df_datafinal[kolom_kriteria].to_numpy()

        vektor_s = np.prod((matrix_x + 1e-9) ** (k * norm_bobot), axis=1)
        vektor_v = vektor_s / np.sum(vektor_s)

        df_wp_result = df_datafinal.copy()
        df_wp_result["Vektor_V"] = vektor_v

        # Membuat DataFrame Ranking
        df_ranking = df_wp_result.sort_values(by="Vektor_V",
ascending=False).reset_index(drop=True)
        df_ranking.index = df_ranking.index + 1

        # 2. Mulai Tampilan Grafik & Data Top 10
        df_top = df_ranking.head(10)

        # Bagian Atas: Tabel Detail Top 10
        st.subheader("📋 Detail Top 10")
        st.write("Laptop dengan nilai preferensi tertinggi berdasarkan
bobot kriteria saat ini.")
        st.dataframe(df_top[["Model", "Vektor_V"]],
use_container_width=True)

        st.markdown("---") # Garis pembatas visual agar rapi

        # Bagian Bawah: Grafik Perbandingan
        st.subheader("📊 Grafik Perbandingan Vektor V")

        fig, ax = plt.subplots(figsize=(12, 6)) # Lebar dinaikkan ke 12
        agar grafik memanfaatkan ruang horizontal

        models = df_top["Model"].values
        scores = df_top["Vektor_V"].values
        y_pos = np.arange(len(models))

        colors = plt.cm.viridis(np.linspace(0.8, 0.3, len(models)))
        bars = ax.barh(y_pos, scores, align='center', color=colors,
edgecolor='white', linewidth=0.5)

        ax.set_yticks(y_pos)
        ax.set_yticklabels(models, fontsize=10)
        ax.invert_yaxis()
        ax.set_xlabel('Skor Preferensi (Vektor V)')
        ax.set_title('Top 10 Laptop Terbaik (Metode WP)', fontsize=14,
fontweight='bold', pad=20)

        for bar in bars:
            width = bar.get_width()
            ax.text(width + 0.0002, bar.get_y() + bar.get_height()/2,

```

```

f'{width:.4f}', va='center', fontsize=10,
fontweight='bold', color='#2E86C1')

ax.spines['right'].set_visible(False)
ax.spines['top'].set_visible(False)

plt.tight_layout()
st.pyplot(fig, use_container_width=True) # Grafik akan otomatis
melebar memenuhi layar

st.markdown("___")

laptop_terbaik = df_ranking.iloc[0] # Mengambil baris pertama
(Peringkat 1)

st.subheader(f"🔍 Analisis Kriteria Rekomendasi Teratas:
**{laptop_terbaik['Model']}**")
st.write("Persentase nilai setiap kriteria (Skala 0-100%). Semakin
tinggi/bagus nilainya, warna bar akan semakin **Hijau**.")

# Proses Normalisasi ke 0-100% menggunakan NumPy berdasarkan nilai
Min-Max dataset
kriteria_label = ["CPU (multiScore)", "GPU (3DMark)", "RAM",
"Storage", "Harga (Price_USD)"]
persentase_kriteria = []

# 1. CPU (Benefit)
val_cpu = laptop_terbaik["multiScore"]
min_cpu, max_cpu = df_ranking["multiScore"].min(),
df_ranking["multiScore"].max()
pct_cpu = ((val_cpu - min_cpu) / (max_cpu - min_cpu + 1e-9)) * 100
persentase_kriteria.append(pct_cpu)

# 2. GPU (Benefit)
val_gpu = laptop_terbaik["3DMark"]
min_gpu, max_gpu = df_ranking["3DMark"].min(),
df_ranking["3DMark"].max()
pct_gpu = ((val_gpu - min_gpu) / (max_gpu - min_gpu + 1e-9)) * 100
persentase_kriteria.append(pct_gpu)

# 3. RAM (Benefit)
val_ram = laptop_terbaik["RAM"]
min_ram, max_ram = df_ranking["RAM"].min(),
df_ranking["RAM"].max()
pct_ram = ((val_ram - min_ram) / (max_ram - min_ram + 1e-9)) * 100
persentase_kriteria.append(pct_ram)

# 4. Storage (Benefit)
val_st = laptop_terbaik["Storage"]
min_st, max_st = df_ranking["Storage"].min(),
df_ranking["Storage"].max()
pct_st = ((val_st - min_st) / (max_st - min_st + 1e-9)) * 100
persentase_kriteria.append(pct_st)

# 5. Harga (Cost -> Rumus dibalik: semakin kecil harga, persentase
semakin tinggi/bagus)
val_pr = laptop_terbaik["Price_USD"]
min_pr, max_pr = df_ranking["Price_USD"].min(),
df_ranking["Price_USD"].max()

```

```

pct_pr = ((max_pr - val_pr) / (max_pr - min_pr + 1e-9)) * 100
persentase_kriteria.append(pct_pr)

# Buat Plot Matplotlib baru untuk kriteria
fig2, ax2 = plt.subplots(figsize=(12, 5))
y_pos_kriteria = np.arange(len(kriteria_label))

# Logika Warna Dinamis: Menggunakan colormap 'RdYlGn'
(Red-Yellow-Green)
# Nilai persentase dibagi 100 agar berada di range 0.0 - 1.0 untuk
matplotlib
colors_kriteria = plt.cm.RdYlGn([p / 100.0 for p in
persentase_kriteria])

bars2 = ax2.barh(y_pos_kriteria, persentase_kriteria,
align='center', color=colors_kriteria, edgecolor='gray', linewidth=0.5)

ax2.set_yticks(y_pos_kriteria)
ax2.set_yticklabels(kriteria_label, fontsize=11,
fontweight='bold')
ax2.invert_yaxis()
ax2.set_xlabel('Persentase Kualitas Kriteria (%)', fontsize=11)
ax2.set_xlim(0, 115) # Beri ruang di kanan untuk label teks

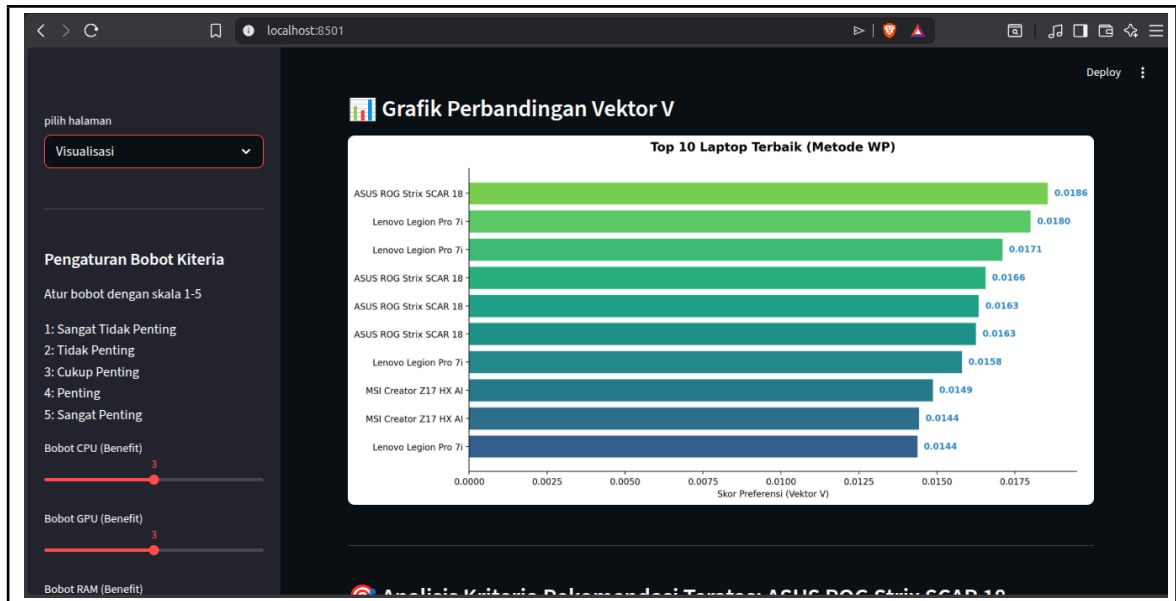
# Tambahkan label teks persentase di ujung bar
for bar in bars2:
    width = bar.get_width()
    ax2.text(width + 1.5, bar.get_y() + bar.get_height()/2,
f'{width:.1f}%', va='center', fontsize=11,
fontweight='bold')

ax2.spines['right'].set_visible(False)
ax2.spines['top'].set_visible(False)
plt.tight_layout()
st.pyplot(fig2, use_container_width=True)

else:
    st.warning("Silakan buka halaman 'Data RAW', 'Data CPU', dan 'Data
GPU' terlebih dahulu untuk memuat dataset ke memori aplikasi.")

```

(tangkapan layar sistem)



BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	Mei 2026			
		Minggu			
		1	2	3	4
1	Pencarian Dataset				
2	Persiapan Dataset				
3	Implementasi WP				
4	Perbaikan Tampilan UI				
5	Pembuatan Laporan Praktikum				

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Ichsan Wahyu Priambudi	123240122	- Persiapan Dataset - Implementasi WP di code
2	Muhamad Ifsal Fadhilah	123240138	- Pencarian Dataset di Keagle - Pembuatan Laporan Praktikum - Perbaikan tampilan UI streamlit

BAB IV KESIMPULAN DAN SARAN

4.2 Kesimpulan

Dari proyek pembuatan Sistem Pendukung Keputusan (SPK) pemilihan laptop *gaming* ini, dapat disimpulkan bahwa metode *Weighted Product* (WP) telah berhasil diimplementasikan secara komputasional untuk memecahkan masalah pengambilan keputusan multi-kriteria. Sistem ini mampu memproses kriteria tipe *cost* (Harga) dan *benefit* (Performa CPU, Performa GPU, RAM, Storage) secara presisi menggunakan perhitungan perpangkatan bobot positif dan negatif.

Keunggulan utama dari rancangan sistem ini terletak pada proses penggabungan data (*data merging*) antara dataset alternatif dengan dataset *benchmark* performa riil, yaitu penggunaan skor *multiScore* untuk pengujian CPU dan skor *3DMark* untuk GPU. Pendekatan ini terbukti menyajikan parameter penilaian yang jauh lebih realistis dan akurat dibandingkan sekadar menggunakan klasifikasi *tier* performa yang statis. Selain itu, pemanfaatan *framework* Streamlit dengan bahasa pemrograman Python berhasil menciptakan antarmuka visual yang interaktif. Hal ini sangat memudahkan pengambil keputusan awam untuk mengatur tingkat prioritas spesifikasi secara dinamis melalui komponen *slider*, sehingga perubahan kalkulasi nilai Vektor V dan hasil perankingan akhir dapat dievaluasi secara instan.

4.3 Saran

Untuk penyempurnaan dan pengembangan sistem di masa mendatang, disarankan agar penelitian selanjutnya melakukan penambahan kriteria spesifik yang krusial bagi kenyamanan *gaming*, seperti tingkat *refresh rate* layar, kualitas sistem pendingin (*cooling system*), dan kapasitas daya tahan baterai.

Selain itu, aplikasi ini dapat diperluas fungsinya dengan menyertakan fitur komparasi perhitungan menggunakan metode pengambilan keputusan lainnya, seperti *Simple Additive Weighting* (SAW), *Analytical Hierarchy Process* (AHP), atau TOPSIS. Penambahan metode pembandingan ini akan berguna untuk menguji tingkat sensitivitas dan keandalan rekomendasi yang dihasilkan. Terakhir, mengingat harga perangkat keras elektronik sangat fluktuatif di pasaran, implementasi teknik *web scraping* atau integrasi *Application Programming Interface* (API) langsung ke toko ritel digital sangat direkomendasikan agar data kriteria harga dapat diperbarui secara *real-time* sesuai kondisi pasar terkini.

DAFTAR PUSTAKA

Malik, S. (2026, Mei 23). *Modern Laptops Hardware & Pricing Dataset 2026*. Kaggle. <https://www.kaggle.com/datasets/sohaibdevv/modern-laptops-hardware-and-pricing-dataset-2026/data>

Alan, X Zhou. (2022, Mei 8). *CPU Benchmarks Compilation*. Kaggle. <https://www.kaggle.com/datasets/alanjo/cpu-benchmarks/data>

Alan, X Zhou. (2022, Mei 8). *GPU Benchmarks Compilation*. Kaggle. <https://www.kaggle.com/datasets/alanjo/gpu-benchmarks>

https://github.com/Mesalllnya/Project_Prak.SCPK